
PRACTICAL - 2

Implementation and Time Analysis of Linear and Binary Search

Aim of the Practical:

To implement Linear Search and Binary Search in C++ and derive their time and space complexities.

2.1 Linear Search

Program:

```
#include <iostream>
using namespace std;
int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
}
int main() {
    int a[] = {10, 25, 7, 40, 15, 30};
    int n = sizeof(a) / sizeof(a[0]);
    int key = 40;
    cout << linearSearch(a, n, key);
}
```

Output:

3

Time Analysis:

Derivation of Time Complexity:

Let n be the number of elements.

Best case: the key is the first element, so only one comparison is required.

$$T_{\text{best}}(n) = c = \Theta(1)$$

Worst case: the key is the last element or absent, so n comparisons are performed.

$$T_{\text{worst}}(n) = c_1 \cdot n + c_2 = \Theta(n)$$

Average case: assuming the key is equally likely to be at any position, expected comparisons are:

$$C_{\text{avg}}(n) = (1 + 2 + \dots + n) / n$$

$$C_{\text{avg}}(n) = [n(n + 1)/2] / n = (n + 1)/2$$

$$T_{\text{avg}}(n) = \Theta(n)$$

Final Time Analysis:

- Best Case: $\Theta(1)$.
- Average Case: $\Theta(n)$.
- Worst Case: $\Theta(n)$.
- Auxiliary Space: $\Theta(1)$.

Result:

Linear Search was implemented successfully and its comparison count was derived for best, average and worst cases.

2.2 Binary Search

Program:

```
#include <iostream>
using namespace std;
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
```



```
int mid = low + (high - low) / 2;
if (a[mid] == key) return mid;
if (a[mid] < key) low = mid + 1;
else high = mid - 1;
}
return -1;
}
int main() {
    int a[] = {5, 10, 15, 20, 25, 30, 35, 40};
    int n = sizeof(a) / sizeof(a[0]);
    cout << binarySearch(a, n, 30);
}
```

Output:

5

Time Analysis:

Derivation using Recurrence / Master Method:

At every step Binary Search discards half of the remaining elements.

$$T(n) = T(n/2) + c$$

Compare with $T(n) = aT(n/b) + f(n)$:

$$a = 1, \quad b = 2, \quad f(n) = \Theta(1)$$

$$n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$$

$$f(n) = \Theta(1) = \Theta(n^{(\log_b a)})$$

Therefore Master Theorem Case 2 applies:

$$T(n) = \Theta(\log n)$$

Equivalently, after k halvings $n/2^k = 1$, so $k = \log_2 n$.

Final Time Analysis:

- Best Case: $\Theta(1)$.
- Average Case: $\Theta(\log n)$.
- Worst Case: $\Theta(\log n)$.
- Auxiliary Space (iterative): $\Theta(1)$.

Result:

Binary Search was implemented successfully, and its logarithmic complexity follows directly from repeated halving.

PRACTICAL - 7

Implementation of Making Change Problem using Dynamic Programming

Aim of the Practical:

To find the minimum number of coins required to make a target amount using dynamic programming and analyze its complexity.

Algorithm:

1. Initialize $dp[0] = 0$ and $dp[1...A] = \infty$.
2. For each amount x from 1 to A , try every coin denomination.
3. If $coin \leq x$ and $dp[x-coin]$ is finite, update $dp[x] = \min(dp[x], dp[x-coin] + 1)$.
4. Return $dp[A]$, or -1 if it is still infinite.

Program:

```
#include <iostream>
#include <vector>
#include <climits>
#include <algorithm>
using namespace std;
int minCoins(const vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;
    for (int x = 1; x <= amount; x++) {
        for (int coin : coins) {
            if (coin <= x && dp[x - coin] != INT_MAX) {
                dp[x] = min(dp[x], dp[x - coin] + 1);
            }
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
int main() {
    vector<int> coins = {1, 3, 4};
    int amount = 6;
    cout << minCoins(coins, amount);
}
```

Output:

2

Step-by-Step DP for coins {1,3,4}, amount 6:

- $dp[0] = 0$
- $dp[1] = 1$ using {1}
- $dp[2] = 2$ using {1,1}
- $dp[3] = 1$ using {3}
- $dp[4] = 1$ using {4}
- $dp[5] = 2$ using {4,1}
- $dp[6] = 2$ using {3,3}

Time Analysis:

Derivation of Time Complexity:

Let A be the target amount and m be the number of coin denominations.

The outer loop runs once for each amount from 1 to A .

Outer-loop iterations = A

For every amount, the algorithm checks all m coin denominations.

Inner-loop iterations per amount = m



Each inner-loop operation performs a constant-time comparison and possible update.

$$T(A,m) = A \cdot m \cdot O(1)$$

$$T(A,m) = \Theta(Am)$$

The dp array contains $A + 1$ entries.

$$S(A) = \Theta(A)$$

Final Time Analysis:

- Time Complexity: $\Theta(Am)$.
- Space Complexity: $\Theta(A)$.

Result:

The Making Change problem was solved successfully using dynamic programming, and the minimum number of coins for amount 6 is 2.